

Introduction to Coding

- **Duration:** One semester — two nine-week terms (18 weeks total)
 - **Primary Language:** Python 3
 - **Supplemental Languages (survey only):** Lua, C++, Java
 - **Prerequisites:** None. Basic computer literacy (file navigation, typing, using a web browser) is assumed.
-

Course Description

This course introduces students to the foundational concepts of computer programming and the algorithmic thinking that underlies all software development. Python 3 will serve as the working language because of its human-readable syntax and operating system agnosticism, allowing students to focus on *how programs are constructed* rather than on language-specific punctuation. Short, structured comparisons with Lua, C++, and Java will be used throughout the course to demonstrate that the underlying concepts — variables, control flow, data structures, functions, and algorithms — are largely language-independent.

By the end of the semester, students should be able to read, write, debug, and reason about small Python programs, and should recognize the same constructs when shown code in another language.

Learning Objectives

Upon successful completion of the course, students will be able to:

1. Explain what a program is and describe, at a high level, how source code is translated and executed by a computer.
2. Write, run, and debug Python programs that use variables, expressions, conditionals, loops, functions, and basic data structures (lists, tuples, dictionaries, strings).
3. Decompose a problem into discrete steps and express the solution as an algorithm using pseudocode and a flowchart before writing code.
4. Implement and explain several foundational algorithms, including linear search, binary search, and at least one elementary sort.
5. Read short code samples in Lua, C++, and Java and identify the equivalent constructs they already know from Python.
6. Use version-control basics (file history, simple commits) and apply common debugging practices, including reading error messages and using `print` / step-through inspection.

7. Demonstrate professional habits: writing readable code, naming things clearly, and commenting where intent is not obvious.

Course Structure

The semester is divided into two nine-week terms. Term 1 establishes the fundamentals in Python. Term 2 extends those fundamentals into algorithms, data structures, and a cross-language survey, ending with a small individual project.

Term 1 — Foundations in Python (Weeks 1–9)

Week	Topic	Key Concepts
1	What is a program?	Hardware vs. software, source code vs. machine code, interpreters vs. compilers, setting up the Python environment and editor
2	Values, variables, and types	Integers, floats, strings, booleans; assignment; basic input/output
3	Expressions and operators	Arithmetic, comparison, logical operators; operator precedence
4	Conditionals	<code>if</code> , <code>elif</code> , <code>else</code> ; boolean logic; truthiness
5	Loops	<code>while</code> and <code>for</code> loops; iteration; <code>break</code> and <code>continue</code>
6	Functions, Part 1	Definition, parameters, return values, scope
7	Strings and lists	Indexing, slicing, common methods, iteration over collections
8	Debugging and reading errors	Tracebacks, common error categories, <code>print</code> debugging, rubber-duck method
9	Term 1 review and checkpoint project	A small program integrating the term's concepts (e.g., a number-guessing game or simple text-based quiz)

Term 2 — Algorithms, Data, and Language Comparison (Weeks 10–18)

Week	Topic	Key Concepts
10	Functions, Part 2	Decomposition, reuse, simple recursion (introductory)
11	Dictionaries and structured data	Key/value storage, lookups, when to use which collection
12	Algorithmic thinking	Pseudocode, flowcharts, problem decomposition, the idea of correctness
13	Searching	Linear search, binary search, why sorted input matters, informal cost comparison
14	Sorting	Bubble sort or insertion sort traced by hand and implemented in Python
15	Files and simple persistence	Reading and writing text files; basic data processing
16	Cross-language survey	The same small program (e.g., FizzBuzz, a function, a loop) shown in Python, Lua, C++, and Java; syntax differences vs. shared concepts
17	Final project work	Student-chosen project from a curated list; planning, implementation, and iteration
18	Final project presentations and review	Brief demonstrations, peer review, semester wrap-up

Cross-Language Component

Each of the four "concept anchors" in Term 1 (variables, conditionals, loops, functions) is revisited briefly with a side-by-side example in at least one other language so that students see the same idea expressed differently. The dedicated cross-language week (Week 16) consolidates this with a single short program implemented in all four languages. The goal is recognition and transferability, not fluency in the additional languages.

Tools and Environment

- **Language runtime:** Python 3 (current stable release).
- **Editor:** Visual Studio Code with the official Python extension.
- **Supplemental language demonstrations:** Provided primarily through instructor-led examples and online sandboxes; no local installation of Lua, C++, or Java is required of students.

- **Take-home work:** All assignments are designed to run on a standard personal/home computer; no specialized hardware is required.

Minimum Computer Requirements

The figures below describe what is required to install and run Python 3 and Visual Studio Code locally and to complete every assignment in this course. Any computer purchased new within roughly the last seven to eight years will comfortably exceed them.

Desktop or Laptop (recommended)

Component	Minimum	Recommended
Operating System	Windows 10 (64-bit), macOS 11 (Big Sur), or a current 64-bit Linux distribution (Ubuntu 20.04 LTS or equivalent)	Windows 11, macOS 13 or newer, Ubuntu 22.04 LTS or newer
Processor	64-bit dual-core, 1.6 GHz	64-bit quad-core, 2.0 GHz or faster (Apple Silicon, recent Intel/AMD)
Memory (RAM)	4 GB	8 GB or more
Free Disk Space	5 GB	10 GB or more
Display	1280 × 720	1920 × 1080 or higher
Input	Physical keyboard and mouse or trackpad	Same
Network	Occasional internet access for installation, documentation, and online sandboxes	Reliable home internet

Chromebooks are acceptable provided they support the Linux development environment (most models released from 2019 onward); otherwise, students on a Chromebook should plan to use the browser-based option described below.

iPad or Other Tablet (acceptable alternative)

A tablet paired with a Bluetooth (or USB-C) keyboard is a workable option for this course, with the understanding that Python is **not** installed directly on the device. Students on a tablet will run their code either through a Python app from the device's app store or through a browser-based Python environment. Both approaches are sufficient for every assignment in the course.

Component	Minimum	Recommended
Device	iPad (8th generation or newer), iPad Air (4th gen or newer), iPad Pro (2018 or newer); recent Android tablets running Android 11+; Microsoft Surface tablets running full Windows	Any current-generation iPad with iPadOS 16 or newer
OS Version	iPadOS 15+, Android 11+, or Windows 10+	Current major release
Storage	64 GB device storage	128 GB or more
Keyboard	External Bluetooth or USB-C keyboard required (on-screen keyboards are not practical for typing code)	Same, plus a stand or keyboard case
Network	Reliable internet connection (required for browser-based Python use)	Same

Software options for tablets:

- **Browser-based (preferred, works on any tablet):** A hosted Python environment such as Replit, Google Colab, or a similar service. This requires only a modern web browser and an internet connection.
- **iPad-native apps:** Apps such as Pythonista, Pyto, or Carnets (Jupyter) provide an on-device Python interpreter and editor and work offline. These are acceptable substitutes for Visual Studio Code on a tablet.
- **Surface tablets running full Windows** can install Python 3 and Visual Studio Code directly and should be treated as laptops for the purposes of this course.

Students using an Android tablet without a full Windows environment should plan to use the browser-based option.

What Students Will Take Away

A student who completes this course should be able to sit down in front of a blank editor, read a simple problem statement, and produce a working Python program that solves it — and, just as importantly, explain *why* their solution works. They should also be able to open a file written in a language they have never formally studied and recognize the basic structure of what they are reading. The intent is not to produce finished software engineers, but to give students a durable mental model of how code works that they can build on in any later course, language, or career path.